

The formal proof for even tori

What is proved, how it is assembled, and what to read

Result. The Lean endpoint proves the paper’s main even-order theorem, with no remaining certificate hypothesis. This guide describes the formalization; the recorded build and endpoint axiom audit are in `VERIFICATION.md`, and the statement-by-statement correspondence with the paper is in `PAPER_CORRESPONDENCE.md`.

The exact mathematical statement

For $q \in \mathbb{N}$, $q > 0$, the board is $(\mathbb{Z}/2q\mathbb{Z})^2$. Two cells attack if their row, column, difference or sum agrees modulo $2q$. Attacks within an army are allowed; every cross-army attack is forbidden. $t(2q)$ is the greatest possible size of two equally large armies. Define, in the notation used by Lean,

$$H_q = \max_{\substack{0 \leq r_0, r_1, c_0, c_1 \leq q \\ r_i, c_i \in \mathbb{N}}} \min\{r_0 c_1 + r_1 c_0, (q - r_0)(q - c_0) + (q - r_1)(q - c_1)\}.$$

Indexing matters: Lean’s `H q` means the paper’s $H(2q)$. In namespace `PeaceableQueens`, `Main.lean` exports

$$\underbrace{t(2q) \leq H_q}_{\text{upperBoundStatement}} \quad \text{and} \quad \underbrace{t(2q) = H_q}_{\text{evenTorusTheorem}}.$$

The propositions are defined in `TheoremStatement.lean` and discharged in `Main.lean`. The former alone is not the completed upper-bound proof. The paper prints `thm:main` as **Theorem 2**; source comments and guides cite stable paper labels.

The argument, in reading order

Step	Meaning and source, relative to <code>PeaceableQueens/</code>
Define the problem	<code>Defs.lean</code> : modular attacks, equal armies, the capped maximum and the finite formula H_q . Maximum-characterization lemmas justify the cap.
Replace armies by line colours	<code>LineColouring.lean</code> : every peaceable pair extends to a colouring of all four line families; conversely, monochromatic cells give peaceable armies after trimming the larger one.
Prove the lower bound	<code>Parity.lean</code> and <code>LowerBound.lean</code> : arbitrary-set parity counting, followed by explicit label choices, gives $H_q \leq t(2q)$.
Normalize configurations	<code>FiniteGeometry.lean</code> and <code>UpperBound/Symmetry.lean</code> : attack-preserving board equivalences and complementation give the canonical chamber.
Bound every order	$q = 1, 2$: <code>SmallBoards.lean</code> . $3 \leq q \leq 129$: checked finite action trees. $q \geq 130$: checked rational branch trees and the local algebraic finish.
Assemble	<code>UpperBound/Assembly.lean</code> partitions all positive q into these three ranges. <code>Main.lean</code> supplies the checked components and combines both inequalities.

Companion references. The paper is `peace_even_torus.tex` (6 September 2026), published with this Lean project in the public repository `jphme/math-problems`, directory `peaceable-queens-even-torus/`. In its `lean/` subdirectory, `FORMAL_PROOF.md` gives obligations S1–S66 and `PAPER_CORRESPONDENCE.md` maps all 66 and all 43 numbered paper results. This PDF presents the argument and its essential interfaces.

The large-order proof: $q \geq 130$

Filenames on this page are relative to `PeaceableQueens/UpperBound/`, abbreviated `UB/` on the next page. The eight normalized line counts are $x = (r_0, r_1, c_0, c_1, d_0, d_1, a_0, a_1)/q$.

1. Geometry gives a feasible rational point. The four row/column parity blocks each have q^2 cells, with 16 colour atoms per block. Normalization, marginal counts and five independent line-pair counts give 40 equations. The two D/A equations aggregate counts across blocks: equal-parity diagonal and antidiagonal labels have two lifts; opposite-parity labels have none. This remains correct when $4 \mid 2q$. No unjustified per-block D/A independence is imposed.

`Incidence.lean`, `BlockMoments.lean` and `ConfigurationMoments.lean` establish these identities. `MomentModel.lean` gives their semantic indexing; `LPModel.lean` uses 95 variables (8 counts, 22 products, 64 atoms, one objective), 42 equalities and 95 inequalities. `ModelCompatibility.lean` checks the flat coefficients; `ConfigurationBridge.lean` proves feasibility with objective $\min(B, W)/q^2$. These implement `lem:inc`, `lem:moments` and the required part of `def:OPT` (S21–S32).

2. Two checked trees localize any possible counterexample. For canonical feasible points, the global tree on $[0, 1]^8$ proves the bound $527/1000$ outside

$$N = [0, \frac{1}{10}] \times [\frac{3}{5}, \frac{7}{8}] \times [0, \frac{1}{10}] \times [\frac{3}{5}, \frac{7}{8}] \times [\frac{19}{20}, 1] \times [0, \frac{1}{10}] \times [\frac{19}{20}, 1] \times [0, \frac{1}{10}].$$

The second tree starts at exactly N and bounds the objective unless

$$\frac{36}{25} \leq x_1 + x_3 \leq \frac{37}{25}, \quad x_0 + x_2 \leq \frac{9}{100}, \quad x_5 + x_7 \leq \frac{9}{200}, \quad x_4 + x_6 \geq \frac{49}{25}.$$

Dual leaves are proved sound by exact linear combinations; splits cover their parent boxes.

`DualLeaf.lean`, `SparseListDualLeaf.lean`, `ScaledIntDualLeaf.lean` and its helper directory prove the arithmetic interfaces. `BranchCertificate.lean` supplies coverage; `Generated/Even/` supplies both complete trees. `CertificateConsequences.lean` connects them to configurations. The escape estimate $527q^2/1000 < H_q$ for $q \geq 64$ forces every hypothetical counterexample into this core. This is `lem:dualleaf`, `lem:branchsound`, `thm:cert1`, `thm:cert2` and `cor:escape` (S33–S39).

3. Integer defects close the localized case. Set $u = r_0$, $R = r_1$, $v = c_0$, $C = c_1$, $e = d_1$, $f = a_1$, $g = q - d_0$, $h = q - a_0$, now as unnormalized counts. Write $\delta = e + f$, $\zeta = g + h$, $Q = 2ef + 2gh$, $\alpha = 2q - R - C - u - v$, $\beta = R + C - q$. The individual support bounds imply

$$\begin{aligned} B &\leq F_B, \quad W \leq F_W, \quad F_B = uv + RC - \beta\zeta + Q, \\ F_W &= (q - u)(q - C) + (q - R)(q - v) - \alpha\delta + Q. \end{aligned}$$

The core gives $Q \leq \alpha\delta$ or $Q \leq \beta\zeta$. In Case A, four explicit integer profile switches absorb the loss and bound the objective by H_q . In the remaining Case B subcase, $gh > 0$ forces $\zeta \geq 2$ because g, h are natural defects. A factored quadratic inequality gives the necessary gap; the exact comparison closes at $q = 130$.

Read `SupportCuts.lean`, `LargeOrderConfiguration.lean`, `LargeOrderAlgebra.lean`, `LargeOrderFinish.lean`, then `LargeOrderCertified.lean`. `ProfileAlgebra.lean` supplies the H_q estimates and escape threshold. These cover `lem:cuts` through `thm:q130` (S40–S54).

The finite proof: $3 \leq q \leq 129$

The search space is the integer box $[0, q]^8$ restricted by the five canonical chamber/budget rows. Each order has a complete action tree. Every action must justify a domain prune, implied endpoint tightening, split into both children, or a terminal support-cut bound. Corner discharge uses a proved multi-affinity condition; interval discharge is independently sound.

UB/FiniteLattice.lean proves the rules and their tree induction. UB/EncodedFiniteCertificate.lean decodes the embedded strings with checked tags, indices, subtrees and end-of-input; malformed data cannot be accepted as a completed proof. The cached and fast implementations have proved equality to this specification: CachedFiniteChecker.lean, FastFiniteChecker.lean, FastFiniteTables.lean, FastCutEval.lean, FastReducedCornerCheck.lean, FastIndependentCorner.lean and IndependentCutMasks.lean, all under UB/.

The threshold is justified, not assumed to equal H_q . The 76 cuts have checked moment-dual witnesses in UB/RecoveredFiniteCutSupport/; UB/FiniteCutTable.lean connects them to the integer polynomials. A tree at a concrete natural bound b proves a cut value at most $12b + 6$. Every cut satisfies $12 \min(B, W) \leq \text{cut value}$, so integrality gives $\min(B, W) \leq b$. An explicit admissible four-count witness separately proves $b \leq H_q$. UB/FiniteCertificateConsequences.lean transports this to $t(2q)$.

There are 127 checked modules, UB/Generated/Finite/Q003.lean through Q129.lean; each supplies acceptance, the H_q witness and upper_bound. UB/Generated/FiniteRange.lean covers the entire interval. These replace the paper's finite ladder and its exceptional even sweeps.

PeaceableQueens/SmallBoards.lean handles $q = 1, 2$ with kernel decisions. On the 4-torus, translate a queen to the origin: only four cells remain available to the other army, and their four triples exclude two armies of size three.

Paper correspondence and exact limits of coverage

Paper result / obligations	What the Lean statements establish
thm:main (Theorem 2); S66	Full equality for every positive even order, with all proof components supplied.
lem:lines, prop:lower; S1–S15	Exact reduction and lower bound. paritySeparated_counts applies to arbitrary row/column sets, with exact homogeneous-cell identities.
lem:Hupper, lem:Hlower; S16–S20	Exact bounds on H_q and explicit IsBig0/Tendsto statements. Corollaries.lean transfers them to $t(2q)$ and states the four displayed numerical values.
def:OPT, thm:cert1, thm:cert2	Rational feasible-point bounds. The real optimum, compactness and attainment are not formalized; actual configuration frequencies are rational.
lem:cuts; S41	Both individual bounds $B \leq F_B$, $W \leq F_W$ are exported in LargeOrderFinish.lean; the minimum bounds used by the finish are their consequences.
lem:boxops, lem:boxsound, prop:ladder; S55–S58	Integer-box soundness and all 127 bounds. No real-box extrema or C++ overflow theorem; Lean uses mathematical integers.
prop:small-even; S64	All numerical conclusions, by action trees for the five historical exceptions, rather than the paper's sweep algorithms.
Odd results, lifting, plaid remark	Odd incidence/results, search reductions, divisor lifting, plaid P calculations and the all-order plaid-plus-two conjecture are outside this formalization. The conjecture is also unproved in the paper.

The smallest source set under the current imports

lean/VERIFICATION_FILES.txt lists every required local file, relative to that directory: **892 Lean sources plus 3 project configuration files**. This is the exact transitive import closure of PeaceableQueens.Main, not a claim that every imported declaration is used by the final proof term.

Required group (relative to the Lean project)	Files
PeaceableQueens/*.lean	8
PeaceableQueens/UpperBound/*.lean	39
PeaceableQueens/UpperBound/ScaledIntDualLeaf/*.lean	7
PeaceableQueens/UpperBound/RecoveredFiniteCutSupport/*.lean	21
PeaceableQueens/UpperBound/Generated/Even/*.lean	689
PeaceableQueens/UpperBound/Generated/Finite/Q*.lean	127
PeaceableQueens/UpperBound/Generated/FiniteRange.lean	1
lakefile.toml, lake-manifest.json, lean-toolchain	3
Total	895

Use the exact manifest, not the globs, when copying the proof. Install the pinned leanprover/lean4:v4.32.2 toolchain and the external dependencies at the revisions in lake-manifest.json (Mathlib v4.32.2). The optional umbrella PeaceableQueens.lean is additionally needed for the project's default target; the manifest uses the explicit Main target. For all exported paper corollaries, COROLLARIES_FILES.txt adds exactly Values.lean, UpperBound/Asymptotics.lean and Corollaries.lean: **898 files, of which 895 are Lean sources**.

Reproduce the build and inspect the endpoint

Also retain four workflow files in scripts/: build_project_serial.py, run_with_memory_limit.sh, AuditUpperBound.lean, check_upper_bound_axioms.py. From lean/, after installing the toolchain:

```
lake exe cache get
python3 scripts/build_project_serial.py --target PeaceableQueens.Corollaries
scripts/run_with_memory_limit.sh lake --rehash build +PeaceableQueens.Corollaries
scripts/run_with_memory_limit.sh lake env lean scripts/AuditUpperBound.lean \
  > .lake/final-upper-bound-audit.log 2>&1
python3 scripts/check_upper_bound_axioms.py .lake/final-upper-bound-audit.log
```

The resource guard uses macOS tools. For other platforms, source-manifest checks and copyable commands, see README.md.

Trust and the public release

The endpoint audit has **3 standard axioms and 483 native computation axioms**: 356 large-tree batches and 127 finite acceptances. The two base-board proofs use kernel decisions. Generic soundness proofs use the standard kernel axioms; scaled-integer dual-leaf checks use kernel decide. Native acceptance additionally trusts Lean's compiler/runtime and evaluated library implementations; see the official proof-validation and axiom documentation.

Generated Lean payloads are required proof inputs. Generators and pinned inputs remain in the working repository; .lake/ stays ignored.

The public release directory peaceable-queens-even-torus/ of jphme/math-problems contains the paper, its ancillary bundle, this guide and the complete Lean closure listed in COROLLARIES_FILES.txt. Continuous integration there rebuilds every hand-written module and a sample of certificate modules on every change; the complete closure is too large for hosted runners and is verified offline as recorded in VERIFICATION.md. lean/scripts/check_paper_values.cpp reproduces the displayed H /plaid values and 200-order histogram. External pinned Lean/Mathlib dependencies are fetched separately.